TDS200 · Kryssplattform

Session 10

Styling with NativeWind & Demo App

NativeWind · Demo app

Cross-Platform Mobile Development

SESSION 10 · OVERVIEW

What you'll learn today

- Step-wise Demo app with basic features
- Style with NativeWind v4 — Tailwind CSS for React Native
- Refactor StyleSheet code to utility classes



Project step

Add NativeWind to your app.



Demo-app

Basic features

SESSION 10 · STRUCTURE

How a 2-hour session runs

0:00-0:05	**Recap & objectives** Where we are in the project; today's goals
0:05-0:45	**Demo app** Basic features
0:45-1:00	**Break** —
1:05-1:40	**NativeWind** NativeWind setup
1:40-1:45	**Assignment brief & Q&A** This week's deliverable, then questions

THEORY · DEMO

Demo app to prepare your exam

- The demo-app is designed to help you prepare for the exam.
- It contains 5 phase branches. phase-1,2,3,4,5, in addition to the main branch.
    - Phase-1 contains basic Tab and Stack navigation. It has Home screen, Profile Screen in Tab navigation. Home Screen and postDetails screen are controlled by Stack navigation.
    - Phase-2 uses Firebase Email/password Authentication to protect the whole app, and auth state is managed by Context.
    - Phase-3 uses Firebase Firestore to manage posts.
    - Phase-4 has added Camera and Image Picker, and uses Firebase Storage to store images.
    - Phase-4 adds user interaction such as like/who has liked.

THEORY · STYLING

Why utility-first styling

- **Tailwind is the most-used CSS framework on the web — NativeWind brings it to React Native.**
  - Style inline with className — no separate StyleSheet object.
  - Consistent spacing/colour scale; less naming, fewer style files.
  - Responsive + dark-mode variants out of the box.
  - Compiles to RN styles — no runtime web CSS.



Faster iteration

Change padding/colour without leaving the JSX.



Consistency

A shared scale keeps the UI coherent across screens.

THEORY · STYLING

Tailwind in React Native

- We can use NativeWind to bring Tailwind CSS classes into React Native / Expo app.
- We then Install Tailwind config by running

  npx tailwindcss init
- This creates a tailwind.config.js. If you have error of not able to deciding which tailwindcss, just create tailwind.config.js manually

terminal

```
npm install nativewind \
  tailwindcss react-native-reanimated \
  react-native-safe-area-context
npx tailwindcss init
```

tailwind.config.js (content)

```
/** @type {import('tailwindcss').Config} */
module.exports = {
content: [
    "./app/**/*.{js,jsx,ts,tsx}",
    "./components/**/*.{js,jsx,ts,tsx}",
],
presets: [require("nativewind/preset")],
theme: {
    extend: {},
},
    plugins: [],
};
```

Install & configure

- Create 3 files at the root folder
    - Create a **babel.config.js** at the root folder, by running npx expo customize babel.config.js.
    - At the root folder of your project, create **nativewind-env.d.ts**, with following content.
    - At the root folder of your project, create **metro.config.js**, with following content.

Metro.config.js (content)

```
const { getDefaultConfig } = require("expo/metro-
config");
const { withNativeWind } = require("nativewind/metro");
const config = getDefaultConfig(__dirname);

config.resolver.unstable_enablePackageExports = true;
config.resolver.unstable_conditionNames = ["react-
native", "require", "default"];
module.exports = withNativeWind(config, { input:
"./global.css" });
```

babel.config.js (content)

```
module.exports = function (api) {
  api.cache(true);
  return {
    presets: [
    ["babel-preset-expo",
    { jsxImportSource: "nativewind" }],
    "nativewind/babel",
    ],
  };
};
```

Nativewind-env.d.ts (content)

```
/// <reference types="expo/types" />
```

SETUP · NATIVEWIND V4

Install & configure

At root folder create global.css (content)

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

At App/_layout.tsx, import global.css

```
import "../global.css";
```

How to use Nativewind at your client app code

```
import { View, Text } from 'react-native';

export default function PostCard() {
  return (
    <View className="p-4 m-2 rounded-2xl
                     bg-white shadow">
      <Text className="text-lg font-semibold
                       text-slate-800">
        Sunset hike
      </Text>
    </View>
  );
}
```

THEORY · STYLING

NativeWind

- After adding NativeWind, clear Expo/Metro cache before starting the app.

  rm -rf .expo .expo-shared node_modules/.cache

- Then restart Expo with a clean cache:

  npx expo start --clear (to clear the cache and restart Expo)

- If the project still has dependency problems, you can reinstall all packages.

  rm -rf node_modules package-lock.json

- Then restart Expo again.

THEORY · STYLING

NativeWind

- Basic Utility classes in Tailwind include:
  - Width and height: i.e., w-1/2, h-12
  - Padding and margin: i.e., p-4, m-2
  - Background color and text color: i.e., bg-blue-500, text-gray-100
  - Font size and weight: i.e., text-xl, font-bold
  - Border width, color and radius: i.e., border-2, border-blue-500, rounded-lg
  - Flexbox: i.e., flex, flex-row, justify-center
  - Grid: i.e., grid, grid-cols-1, gap-4
  - State (hover, focus, active, disabled):i.e., focus:outline-none, active:bg-blue-200
  - Responsive design: i.e., md:bg-red-500, lg:grid-cols-4
- Tailwind (nativewind) can coexist with react native stylesheet.

ASSIGNMENT 10

NativeWind & Jest

YOUR TASKS

- In this session, `StyleSheet` has been replaced with NativeWind `className` on the home screen and profile page. Your task is to do the same for `components/Post.tsx` — the card that appears in the feed.

DELIVERABLE

A NativeWind-powered app with consistent styling applied across all major screens.

EST. EFFORT

≈ 0.5 hours